

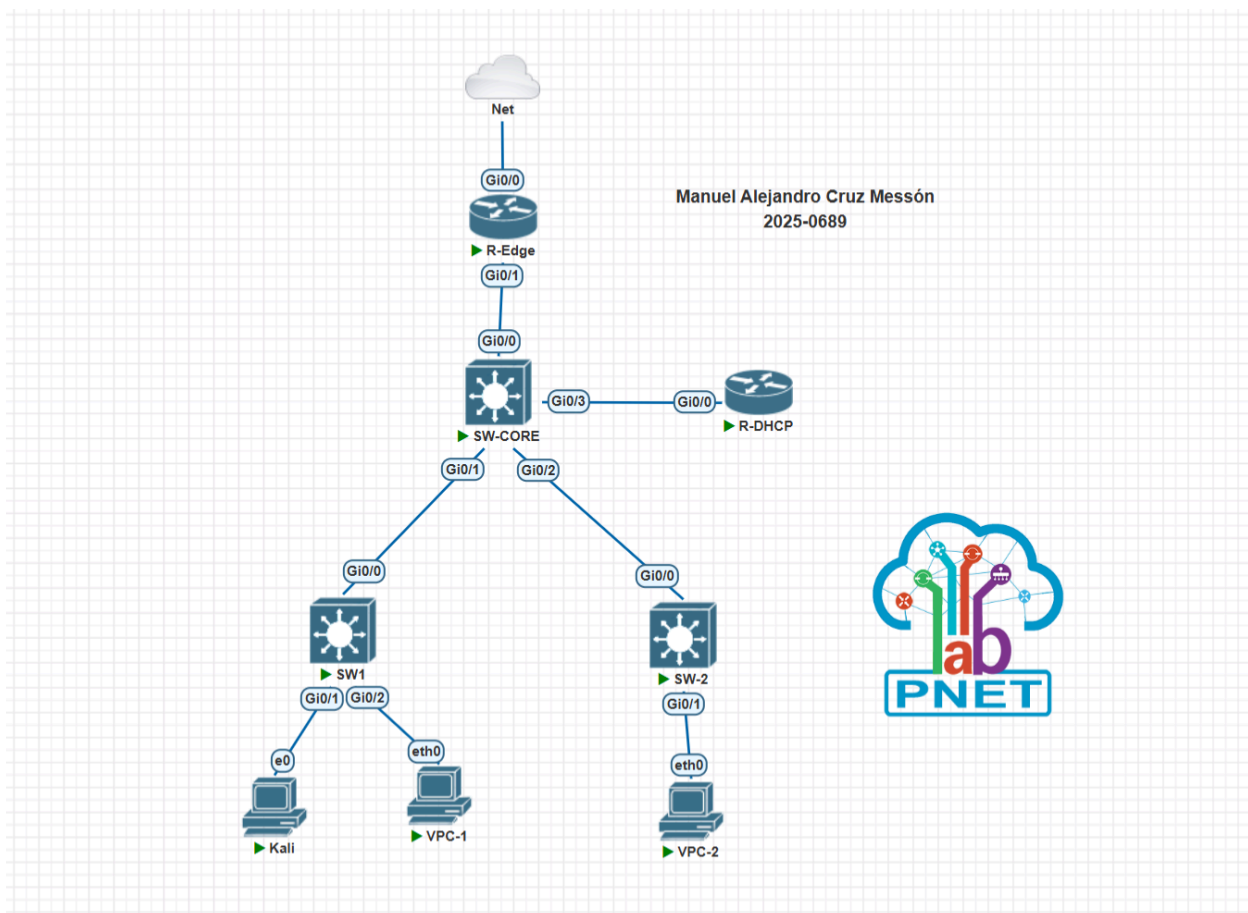
## STP Claim Root Attack:

### 1. Objetivo del Laboratorio

El objetivo fundamental de este laboratorio es comprender y evaluar la seguridad del protocolo **STP (Spanning Tree Protocol - IEEE 802.1D)** en la capa de enlace de datos (Capa 2). El ejercicio práctico permite analizar la vulnerabilidad estructural de las redes locales cuando los conmutadores (Switches) aceptan mensajes de control sin autenticación previa. Mediante este laboratorio, se demuestra cómo un dispositivo no autorizado puede alterar la topología de la red, forzar una re-convergencia global y posicionarse de manera estratégica para interceptar o interrumpir el flujo de datos (ataques de denegación de servicio o Man-in-the-Middle).

### 1. Topología de la red

La topología representa una red de laboratorio estructurada bajo una arquitectura jerárquica simple, donde todos los dispositivos internos coexisten en la VLAN 89. La red cuenta con servicios automáticos de asignación de direccionamiento IP (DHCP) administrados por un enrutador dedicado, y salida a redes externas (Internet) a través de un enrutador de borde con traducción de direcciones



## A. Hardware y Dispositivos

La infraestructura física y los nodos que componen la topología se distribuyen según sus roles funcionales en la red:

- **Dispositivos de Enrutamiento (Capa 3):**
  - **R-Edge:** Enrutador de borde perimetral encargado de la salida a redes externas.
  - **R-DHCP:** Enrutador dedicado exclusivamente a la administración y distribución de direccionamiento IP dinámico en la red local.
- **Dispositivos de Conmutación (Capa 2):**
  - **SW-CORE:** Switch central (Núcleo) que interconecta los enrutadores y distribuye el tráfico hacia los switches de acceso.
  - **SW-1 y SW-2:** Switches de acceso encargados de proveer conectividad directa a los nodos finales.
- **Dispositivos Finales (Hosts):**
  - **Kali:** Estación de trabajo orientada del atacante.
  - **VPC-1 y VPC-2:** Computadoras virtuales de escritorio (Virtual PCs) que actúan como usuarios finales de la red.
  - **Net:** Nube que simula el entorno de red externa o Internet.

## B. Componentes de Software

Entorno lógico y sistemas operativos que corren sobre la infraestructura:

1. **Sistemas Operativos de Red:** Software basado en emulación de Cisco (IOS) para la gestión y ejecución de protocolos de red (CDP, DHCP, NAT, Routing) en los routers y switches.
2. **Sistemas Operativos de Hosts:**
  - **Kali Linux** instalado en la estación atacante.
  - OS ligero (**VPCS**) en las terminales de usuario para pruebas de conectividad básica (Ping, Traceroute).

## C. Segmentación y Parámetros de Red

Definición del direccionamiento lógico, segmentación LAN y salida a Internet:

- **Segmento de Red Interno:** 192.168.89.0/24 (Máscara de subred 255.255.255.0).
- **VLAN Configurada:** VLAN 89, segmento único donde coexisten de forma nativa todos los dispositivos internos, switches (vía SVI) y routers.
- **Puerta de Enlace (Default Gateway):** 192.168.89.254 (Configurada en la interfaz Gi0/1 de R-Edge). Es el nodo encargado de recibir todo el tráfico interno con destino externo y realizar NAT/PAT para darle salida hacia Internet.

#### D. Interfaces Utilizadas

| Dispositivo Origen | Interfaz Local | Dispositivo Destino | Interfaz Remota |
|--------------------|----------------|---------------------|-----------------|
| <b>R-Edge</b>      | Gi0/0          | Net (Nube)          | —               |
|                    | Gi0/1          | SW-CORE             | Gi0/0           |
| <b>R-DHCP</b>      | Gi0/0          | SW-CORE             | Gi0/3           |
| <b>SW-CORE</b>     | Gi0/0          | R-Edge              | Gi0/1           |
|                    | Gi0/3          | R-DHCP              | Gi0/0           |
|                    | Gi0/1          | SW1                 | Gi0/0           |
|                    | Gi0/2          | SW2                 | Gi0/0           |
| <b>SW-1</b>        | Gi0/0          | SW-CORE             | Gi0/1           |
|                    | Gi0/1          | Kali                | e0              |
|                    | Gi0/2          | VPC-1               | eth0            |
| <b>SW-2</b>        | Gi0/0          | SW-CORE             | Gi0/2           |
|                    | Gi0/1          | VPC-2               | eth0            |
| <b>Kali</b>        | e0             | SW1                 | Gi0/1           |
| <b>VPC-1</b>       | eth0           | SW1                 | Gi0/2           |
| <b>VPC-2</b>       | eth0           | SW2                 | Gi0/1           |

## 2. Objetivo del Script

El script stp.py es una herramienta ofensiva automatizada programada en Python (compatible con Scapy 2.5.0) cuyo propósito es **forzar la elección de la máquina atacante como el "Root Bridge" (Punto Raíz)** de la topología STP. Sus metas técnicas específicas son:

- **Inyección de BPDUs Modificadas:** Construir de manera artesanal y a bajo nivel paquetes BDPUs (Bridge Protocol Data Units) de configuración legítimos en su sintaxis pero maliciosos en su contenido.
- **Manipulación de Criterios de Elección (Prioridad Superior):** Configurar el parámetro de prioridad del puente en 0 (el valor numérico más bajo y, por ende, la prioridad más alta elegible en STP) para obligar a los switches legítimos de la red a ceder el rol de Root Bridge.
- **Suplantación de Identidad Física (MAC Spoofing):** Permitir al operador definir una dirección MAC personalizada (--mac) dentro del identificador del puente (Bridge ID) para evadir filtros básicos basados en la dirección física real del atacante.
- **Mantenimiento del Ataque (Sostenibilidad):** Enviar ráfagas continuas de BPDUs respetando el ciclo estándar de Hello Time (2 segundos por defecto) para evitar que los switches reales asuman la pérdida del Root Bridge y restauren la topología legítima.

## 3. Parámetros Usados

El script implementa el módulo argparse para flexibilizar el ataque desde la consola de comandos, exponiendo los siguientes parámetros:

- **-i, --interface (Obligatorio):** Especifica la tarjeta de red (ej. eth0, ens33) sobre la cual se inyectarán las tramas hacia el switch de acceso.
- **-p, --priority (Opcional, por defecto 0):** Define el valor de prioridad que se le asignará al Bridge ID falso. El script valida obligatoriamente que sea un múltiplo de 4096 (debido al estándar del identificador de sistema extendido).
- **--mac (Opcional):** Dirección MAC hexadecimal personalizada (ej. 00:11:22:33:44:55) para la suplantación dentro de la BDPDU. Si no se especifica, el script extraerá la MAC real de la interfaz.
- **--interval (Opcional, por defecto 2):** Define el tiempo de espera en segundos entre el envío de cada trama BDPDU.
- **-c, --count (Opcional, por defecto 0):** Cantidad exacta de paquetes a inyectar antes de finalizar automáticamente el ataque (el valor 0 representa un bucle infinito).

#### 4. Requisitos para Utilizar la Herramienta

Para la ejecución correcta del script en el escenario de pruebas, el entorno operativo debe cumplir con los siguientes requisitos:

- **Privilegios de Superusuario (Root):** El script ensambla de forma binaria los encabezados e inyecta tramas directamente a nivel de Capa 2 (usando sockets crudos mediante la función `sendp()` de Scapy), acción restringida a usuarios con privilegios elevados (`sudo`).
- **Sistema Operativo Linux:** La manipulación de descriptores de socket crudos a nivel Ethernet y la importación de librerías para la resolución de hardware local dependen del subsistema de red de Linux.
- **Librería Scapy v2.5.0 y Módulo struct:** Se requiere Scapy para la transmisión final de las tramas y el módulo nativo `struct` para empaquetar los datos en formato binario de red (Big-Endian).
- **Puerto de Acceso Activo:** El host atacante debe estar conectado a una interfaz de switch donde el protocolo de árbol de expansión (STP) esté habilitado y procese tramas de control entrantes.

#### 5. Documentación del Funcionamiento del Script

El script opera bajo una lógica de inyección de bajo nivel secuencial detallada a continuación:

##### Fase 1: Validación y Captura de Parámetros

Al iniciar, el bloque `main()` evalúa que el script corra como root (`os.getuid() != 0`) y procesa los argumentos.

Valida matemáticamente que la prioridad sea un múltiplo de 4096 (exigencia del protocolo para acomodar el ID de la VLAN en los 12 bits inferiores del campo de prioridad).

##### Fase 2: Ensamblado Estructurado del Paquete (Función `create_stp_packet`)

A diferencia de otros scripts que confían en las capas automatizadas de Scapy, este código empaqueta la estructura binaria de forma exacta usando el módulo `struct` (`struct.pack` con el modificador `!` para forzar el ordenamiento binario de red):

3. **Identificador de Puente (Bridge ID / Root ID):** Combina los 2 bytes de la prioridad configurada con los 6 bytes de la dirección MAC (ya sea la real o la personalizada) para formar los campos críticos de 8 bytes que definen al Root ID y Bridge ID.
4. **Payload STP:** Une los campos requeridos por el estándar IEEE 802.1D: Protocol ID (0), Version (0), BPDU Type (0 para tramas de configuración), Flags (0), Root Path Cost (0 para declarar que él mismo es la raíz), Port ID (0x8001), Message Age (0) y los temporizadores reglamentarios (Max Age, Hello Time, Forward

Delay).

5. **Encabezado LLC (Logical Link Control - IEEE 802.2):** Se antepone la cadena de bytes \x42\x42\x03 correspondientes a DSAP (Destination Service Access Point), SSAP (Source Service Access Point) fijados en 0x42 (STP) y el campo de control 0x03 (Unnumbered Information).
6. **Encabezado Ethernet:** Corona el paquete fijando como dirección física de destino la dirección MAC multicast estricta de STP: 01:80:C2:00:00:00.

### Fase 3: Bucle de Inyección Continua (Función start\_attack)

- El hilo principal arranca un ciclo infinito (while True) regulado por la función time.sleep().
- Cada 2 segundos (intervalo por defecto), invoca a send\_stp\_packet(), la cual encapsula la estructura binaria cruda en un objeto genérico Ether() de Scapy para forzar la salida directa a través de la interfaz seleccionada por medio de sendp().
- Este envío constante engaña a los switches vecinos, quienes al recibir una BPDU con un Bridge ID numéricamente inferior (Prioridad 0), actualizan sus tablas de estado, declaran al puerto del atacante como el nuevo "Root Port" y comienzan a retransmitir la topología alterada al resto de la infraestructura de la red local.

## 6. Documentación de Contra-medidas

Para blindar una red conmutada frente a manipulaciones de la topología STP, se deben configurar de manera obligatoria las siguientes protecciones perimetrales en los switches:

- **STP BPDU Guard (Protección de BPDU):** Es la contramedida más eficaz y recomendada para implementar en los puertos de acceso (interfaces orientadas a usuarios finales o dispositivos finales como impresoras y servidores).
  - Mecanismo: Si un puerto tiene habilitado BPDU Guard e intercepta un solo mensaje BPDU (como las tramas generadas por este script), el switch asume de inmediato una violación de seguridad o una anomalía de red, cambia el estado del puerto a deshabilitado por error (err-disable) y corta todo el tráfico eléctrico del atacante de forma instantánea.
- **STP Root Guard (Protección de la Raíz):** Se configura exclusivamente en los puertos de switches de distribución o núcleo que apuntan hacia switches secundarios o de acceso donde **no** debería residir jamás el Root Bridge legítimo.
  - Mecanismo: Si el script inyecta una BPDU con prioridad superior (Prioridad 0) a través de una interfaz protegida con Root Guard, el switch interceptará el paquete y, en lugar de aceptar la nueva topología, transicionará ese puerto específico a un estado intermitente denominado "root-inconsistent" (raíz inconsistente). En

este estado, el switch descarta el tráfico de datos y de control de esa interfaz, protegiendo la identidad del Root Bridge real. En cuanto el script deja de transmitir, el puerto se recupera automáticamente.

- **PortFast:** Característica complementaria que salta los estados intermedios de STP (Listening y Learning) en puertos de acceso para acelerar la conectividad. Al asociarse con BPDU Guard, garantiza que los terminales de usuario no puedan alterar bajo ninguna circunstancia el árbol de expansión de la organización.